CSC501 C++ for Engineers
Semester 2 – 2025

Lab Exercise 1

Tutorial Questions

1. Explain how and why machine languages evolved into assembly language and later into high-level languages such as Basic, FORTRAN, C++, and Java.
2. Explain the role of a compiler in high-level languages.
3. Describe the following terms: programs, programmers, and programming languages.
4. Is it necessary for the programmer to meet with the user? Why or why not? Explain your answer.
5. What are the primary differences between computer programmers and software engineers?
6. Describe how an assembler and a compiler are similar.

Practical Exercise

Task 1

You will familiarize yourself with DEV C++ IDE and write the very first C++ program.

1. Start Dev-C++ IDE program. Go to START -> Programs -> Dev-C++ ->
2. Create a **new source** file by clicking on toolbar button.
3. Save the file at an appropriate location by choosing File -> Save As. Organise your files properly in folders.
4. Type the following C++ code. You should write the code exactly as follows:

```
#include <iostream>
using namespace std;

int main()
{
	cout << "First Programming Class" << endl;
	return 0;
}
```

5. Compile the program by choosing Execute -> Compile (or F9). If you have errors try removing the errors.
6. If there are no errors then run the program by choosing Execute -> Run (or F10).

   Note that you can combine steps 6 and 7 by pressing F11.

Task 2

Pumpkin Entertainment Center has requested you to produce a program that outputs an inventory of its movies to the screen. The following screen output is an example of how the output should look like:

```
        ***********************************************
                Pumpkin Entertainment Center
                        Video Inventory
        ***********************************************

Title                   Category        Format          Quantity
-----                   --------        ------          --------

Sixth Sense             Romance         DVD             1
Kangaroo Jack           Comedy          DVD             2
Original Sin            Romance         VCD             3
Identity                Suspense        TAPE            2
Gothika                 Horror          VCD             1
They                    Horror          DVD             2
Die Hard                Action          TAPE            1
Johnny English          Action          VCD             1


Press any key to continue . . .
```

1. Use the following skeleton code:

```
#include <iostream>
using namespace std;

int main()
{
      // write your codes here

      return 0;
}
```

2. Following codes can be used to generate the company title:

```
cout << "\n\t***********************************************\n";
cout << "\t\tPumpkin Entertainment Center" << endl;
cout<< "\t\t\tVideo Inventory" << endl;
cout << "\t***********************************************\n\n";
```

3. Similarly using **cout**, **\n**, **\t** and **endl**, given in the code above, write the rest of the output statements to produce the required text.

Task 3

The purpose of this exercise is to produce a catalogue of typical syntax errors and error messages that will be encountered by a beginner and to continue acquainting you with the programming environment. This exercise should leave you with knowledge of what error to look for when given any number of common error messages.

Deliberately introduce errors to the program, compile, record the error and the error message, fix the error, compile again (to be sure you have the program corrected), then introduce another error. Keep the catalog of errors and add program errors and messages to it as you continue through this course.

The sequence of suggested errors to introduce is:

1. Put an extra space between the **<** and the **iostream** file name.
2. Omit one of the **<** or **>** symbols in the include directive.
3. Omit the int from the **int main()**.
4. Omit or misspell the word **main**.
5. Omit one of the **()**, then omit both the **()**.
6. Continue in this fashion, deliberately misspelling identifiers (**cout**, **system**, and so on).
7. Omit one or both of the **<<** in the **cout** statements; leave off the ending curly brace **}**.